

PatchPilot — CLAUDE.md

Exploit-to-Remediation Automation Platform · EU SaaS · GDPR + NIS2 Read this file at the start of every session. Update the "Current State" section after every session.

What PatchPilot Is

PatchPilot is a governance-native patch management platform for 500–5,000 endpoint Windows fleets.

Core promise: Fastest path from exploitation signal (CISA KEV, MSRC advisory) to verified remediation, with append-only audit and reproducible decisions by default.

Market position: Companies too large for Intune-only hygiene, too small for Tanium-scale spend.

Category: Exploit-to-Remediation Automation Platform. Not a patch manager. Not a scanner.

The Six-Stage Loop

Signal → Scope → Decide → Execute → Verify → Evidence

- **Signal:** KEV/MSRC/EPSS/NVD/GHSA ingest
- **Scope:** Map advisory to affected endpoints via KB baseline + normalized software inventory
- **Decide:** Urgency score + org policy gates → auto-deploy or notify+approve with evidence
- **Execute:** Ring rollout, idempotent state machine, reboot contract, winget→relay→direct fallback
- **Verify:** Post-install KB check + app version confirmation, stale baseline flagged
- **Evidence:** Audit log, MTTRem metric, admin notifications with facts

Zero-Day Off-Ramp: When no patch exists → `UnpatchedExposure` entity (separate state machine, recheck loop, mitigation evidence trail). Loop never exits just because nothing to

install.

Architecture

Runtime Split (three separate environments — never combine)

Runtime	Service	Platform	Region
Web	Next.js dashboard	Vercel	Edge (no data)
API	FastAPI + uvicorn	Fly.io patchpilot-api	Frankfurt fra
Worker	Celery workers	Fly.io patchpilot-worker	Frankfurt fra
Beat	Celery Beat scheduler	Fly.io patchpilot-beat	Frankfurt fra (1 instance only)
Database	PostgreSQL	Supabase Frankfurt	EU-West
Storage	Object storage	Supabase Storage Frankfurt	EU-West
Auth	Dashboard user auth	Supabase Auth	EU-West
Queue	Redis	Upstash Frankfurt	EU-West

Hard boundary: Vercel (Next.js) talks only to `api.patchpilot.com` . Never to Supabase directly from the browser. Agents never talk to Supabase — only to `api.patchpilot.com` .

Domain Layout

app.patchpilot.com

api.patchpilot.com

downloads.patchpilot.com

→ Vercel (Next.js dashboard)

→ Fly.io patchpilot-api

→ Supabase Storage via Vercel CDN rewrite

Stack

- **Backend:** FastAPI + SQLAlchemy async + PostgreSQL (Supabase) + Celery + Redis (Upstash)

- **Migrations:** Alembic
 - **Agent:** Python + PyInstaller (Windows: pywin32 for WUA/registry access)
 - **Signing:** Authenticode EV (Windows) via CI pipeline — DigiCert KeyLocker or Azure Trusted Signing
 - **Frontend:** Next.js (Vercel) — API-first, no direct Supabase client calls
-

CRITICAL SECURITY INVARIANTS — never violate these

1. **Backend network:** API is only reachable through Fly.io proxy. No direct Postgres access from outside.
2. **mTLS header guard:** `HardHeaderStrip` ASGI wrapper strips `X-Device-Cert-CN` and `X-Device-Cert-Fingerprint` from ALL requests unconditionally at ASGI layer before `MTLSHeaderGuard` runs. Never remove this.
3. **Device auth:** `get_mtls_device()` validates `cert_fingerprint` against DB on every agent request. Tier 1: Redis revocation check. Tier 2: DB query. Never skip either tier.
4. **Revocation cache:** `warm_revocation_cache_on_startup()` runs at every server start.
5. **Audit log INSERT-only:** App DB user (`patchpilot_app`) has `INSERT` only on `audit_log`. `UPDATE` and `DELETE` are revoked at DB level. Never add a cleanup job to `audit_log`.
6. **Agent auth = mTLS only.** Agents do NOT use JWT. Device identity is mTLS cert only. These two auth paths must never intersect.
7. **CA private keys NEVER stored in plaintext.** Always AES-256-GCM encrypted with `PKI_MASTER_KEY` (Fly.io secret, never DB). DB stores `ca_key_encrypted` `BYTEA`. Only `patchpilot_pki` DB role may read it. Column `ca_key_pem` does not exist.
8. **All intel feed fetches store raw blob BEFORE parsing.** Parse errors store `parse_error` status and alert ops. Never silently lose provenance.
9. **NVD is enrichment only.** Never an emergency trigger. Cadence: 6h intentional.
10. **Audit critical events BLOCK.** If `audit_log` `INSERT` fails for a critical event, raise `AuditInsertError` and the action must NOT proceed. Critical events: `policy.evaluated`, `deployment.dispatched`, `cert.revoked`, `cert.enrolled`, `ai.external_call`, `ai.blast_radius_override`, `exposure.accepted_risk`, `user.role_changed`, `org.settings_changed`.

11. **Enrollment tokens = two-part format.** `token_id` (public, indexed) + `token_secret` (bcrypt'd). Client sends both. Server fetches by `token_id` ($O(1)$), verifies one bcrypt hash. Never table-scan `enrollment_tokens`.
12. **Long-poll = Redis BLPOP.** Not a 2s polling loop. Dispatch = `rpush f"commands: {device_id}"`. Timeout = 55s. Returns `{command: null}` on timeout, not 204.
13. **Parse errors $\neq$ availability failures.** Parse errors indicate schema drift. Store blob, mark `parse_error` status, alert ops. Do NOT serve `last_good_data` on parse error.
14. **Zero-day RESPONSE, not detection.** Use this wording in all code, comments, API responses, and any generated copy. PatchPilot responds to published signals. It does not detect novel threats.
15. **EU data residency enforced.** All Supabase, Fly.io, Upstash resources in Frankfurt. Anthropic API only when `org.settings.ai_policy.external_ai_enabled = true` (explicit org consent). Default: AI disabled.
16. **Tenant isolation:** Every customer-data table has `org_id NOT NULL`. Every router uses `get_org_scope()`. Every DB query includes `WHERE org_id = scope.org_id`. Cross-org leak = existential incident.

Key Decisions Log

Decision	Rationale
mTLS replaces JWT for device auth	Long-lived tokens = breach amplification
NVD cadence: 6h (not 2h, not hourly)	Enrichment only. MSRC+KEV carry urgency. 6h avoids rate limits.
KB collection: WUA primary $\rightarrow$ registry+DISM fallback $\rightarrow$ stale cache	Single-method risk: WUA can hang >30s
Winget: scheduled task workaround for SYSTEM context	Winget needs NETWORK SERVICE context, not SYSTEM
Revocation: Redis fast-path $\rightarrow$ DB authoritative	Redis fail $\rightarrow$ DB, never silent pass
Zero-day: <code>UnpatchedExposure</code> is first-class entity	NULL check is not a workflow. Entity has state machine.
MTTRem: <code>signal_ingested_at</code> =	Measures from when system knew, not

min(kev_added_date, published_at)	when admin knew
Envelope encryption for CA keys	DB compromise ≠ trust model collapse
BLPOP for command delivery	60,000 Redis ops/min wasted with 2s polling at 2000 endpoints
Inventory delta via section hashes	480K full payloads/day mostly zero-change at 2000 endpoints
Telemetry snapshot per job	Phase 3 anomaly detection needs data collected in Phase 1
Pinned parser contract tests	Feed schema drift is silent without them
Supabase Auth for dashboard, mTLS for agents	Clean separation, never mix
Fly.io Frankfurt for API and workers	Long-running workers, EU region, mTLS termination
GDPR opt-in gate for AI	Legal mechanism: explicit consent before EU→US data transfer
NIS2 incident table with deadline columns	Essential-sector customers legally require this
Ship Phase 1–3 before Phase 4 AI	The wedge is KEV→ring→MTTRem, not AI narration

Secret Names (all environments — never commit values)

```
# Platform
SUPABASE_URL
SUPABASE_SERVICE_KEY      # server-side only, never NEXT_PUBLIC_
SUPABASE_JWT_SECRET        # for FastAPI JWT validation
DATABASE_URL               # Supabase PgBouncer pooled connection string

# Queue
REDIS_URL                  # Upstash Frankfurt

# PKI — NEVER in DB
PKI_MASTER_KEY             # 32 bytes, hex-encoded (64 chars). Fly.io secret o
```

```
# Intel feeds
MSRC_API_KEY
NVD_API_KEY

# AI (optional, org-level gate)
CLAUDE_API_KEY          # Only used when org.settings.ai_policy.external_ai

# CI/CD signing (GitHub Actions secrets only)
SIGNING_CERT_P12         # base64-encoded EV cert
SIGNING_CERT_PASSWORD

# Internal
SECRET_KEY               # FastAPI session secret
```

Database Role Model

```
-- Three DB roles
patchpilot_app  -- application user: SELECT/INSERT/UPDATE/DELETE on most tables
                  -- EXCEPTION: INSERT only on audit_log (no UPDATE or DELETE)
                  -- EXCEPTION: no SELECT on org_cas.ca_key_encrypted

patchpilot_pki  -- PKI service only: SELECT on org_cas.ca_key_encrypted
                  -- Used only by OrgCA service when signing certs

patchpilot_ro   -- read-only: for analytics, reporting, compliance exports
```

Feed Cadence (settled — do not change)

Feed	Cadence	Role	Trigger?
CISA KEV	Every 4h	Emergency trigger	YES — KEV entry queues fleet check
MSRC	Every 2h	Emergency co-trigger (Windows)	YES — OOB advisory triggers matching
EPSS	Daily 02:00	Urgency multiplier	NO
NVD	Every 6h	Enrichment only	NO

GitHub Advisory	Daily 03:00	Third-party coverage	NO
Vendor advisories	Daily 04:00	Lag reduction	NO

MSRC resilience: 5x exponential backoff (BASE=30s, MAX=600s), Retry-After header, 2h Redis cache per monthly release, `intel_last_good` fallback, 12h staleness threshold (CRITICAL).

Data Model Key Tables

organizations	- org_id, name, settings JSONB (ai_policy, patch_policy, data
users	- org_id, email, password_hash, role: org_admin admin viewer
devices	- org_id, dept_id, hostname, os_build, cert_fingerprint, cert cert_revoked_at, criticality, tags, last_seen_at, inventory winget_available, relay_node_available
enrollment_tokens	- token_id (public, unique indexed), token_hash (bcrypt of se org_id, dept_id, expires_at, max_uses, used_count, label
org_cas	- org_id, ca_cert_pem, ca_key_encrypted BYTEA (AES-256-GCM),
audit_log	- id, timestamp, org_id, user_id, device_id, event_type, reso changes JSONB, ai_feature, ip_address, result [INSERT only for patchpilot_app role]
vulnerabilities	- cve_id, cvss, epss_score, in_cisa_kev, kev_added_date, publ
advisories	- MSRC/vendor bulletins + advisory_vulnerabilities join
remediations	- fix artifacts (KB numbers) + playbook JSONB (type: patch co + remediation_vulnerabilities + remediation_os_targets
device_vulnerabilities	- device_id, vuln_id, org_id, status, urgency_score, signal_ingested_at, patched_at, mttrem_hours GENERATED (patched_at - signal_ingested_at
unpatched_exposures	- vuln_id, org_id, affected_count, mitigations JSONB, status: open mitigated patched accepted_risk, recheck_interval INTERVAL DEFAULT '1 hour'
deployment_jobs	- org_id, device_id, remediation_id, ring, state, state_updat playbook_snapshot JSONB, telemetry_before JSONB, telemetry_
intel_feed_blobs	- feed_source, fetched_at, blob_hash, storage_path (Supabase status: pending processed parse_error, error
intel_feed_health	- feed_source PK, last_fetched_at, last_success_at, consecuti parse_failures, last_parse_error, is_stale
intel_last_good	- feed_source PK, data BYTEA, stored_at

```
deletion_requests  - org_id, user_id, request_type, requested_at, completed_at,
nis2_incidents     - org_id, title, severity, detected_at,
                    early_warning_due GENERATED (detected_at + 24h),
                    notification_due  GENERATED (detected_at + 72h),
                    final_report_due  GENERATED (detected_at + 3 months)
```

Agent Architecture

Inventory delta model:

- Agent hashes each section (apps , kbs , os) independently
- Sends only changed sections each check-in
- Local hash cache: C:\ProgramData\PatchPilot\inventory_hashes.json
- Daily forced full send at 03:00 local time regardless of hash

KB collection — dual method:

1. WUA COM API (authoritative, cap 2000 entries, timeout-guarded)
 2. CBS registry + DISM /get-packages (fast, deterministic)
 3. Local JSON cache fallback (stale after 8h)
- Payload includes collection_method , stale bool, cached_at timestamp
 - Server annotates vulnerability matches as “uncertain KB baseline” when stale=true

Winget — three-path fallback:

1. winget upgrade --silent (if winget_available from last check-in)
 2. Relay node download (if relay_node_available)
 3. Direct MSI download with SHA256 verification
- SYSTEM context workaround: schtasks to run winget as NETWORK SERVICE

Telemetry snapshot (per job, before + after):

- cpu_percent_1s (psutil)
- crash_events_24h (Event Log: IDs 6008, 1001, 41)
- reboots_7d (Event Log: IDs 6009, 1074)

- `system_disk_free_gb` (psutil)
- Stored in `deployment_jobs.telemetry_before/after` JSONB

Job State Machine

```

QUEUED → DOWNLOADING → INSTALLING → PENDING_REBOOT → VERIFYING → COMPLETE
                                     ↓
                                USER_DEFERRED (max 3, 4h window)
                                     ↓
                                FORCED_REBOOT (after 24h)
FAILED → DIAGNOSING → AUTO_RETRY (max 3) | FAILED_FINAL

```

- Invalid transitions raise `InvalidTransition` — never silent fail
- Audit log written BEFORE state change
- Idempotency rules per state (downloading=restart-safe, installing=check-first, verifying=idempotent)

Ring Rollout

Ring	Size	Halt threshold
Canary	1% or 3 devices (whichever larger)	>20% failure → auto-halt
Pilot	10% of fleet	Requires canary pass
Broad	Remainder	Requires pilot pass

Anomaly halt thresholds (from telemetry): `cpu_percent > 80` , `crash_events_24h > 2` ,
`reboots_7d > 5` , `disk_free < 3GB`

MTTRem — Flagship KPI

```

-- Core columns
signal_ingested_at TIMESTAMPTZ -- populated by intel fetcher: min(kev_added_date
patched_at TIMESTAMPTZ         -- set by verification step

```

```
mttrem_hours GENERATED ALWAYS AS (  
    EXTRACT(EPOCH FROM (patched_at - signal_ingested_at)) / 3600  
) STORED  
  
-- API endpoint  
GET /api/v1/orgs/{id}/metrics/mttrem?period=30d&group_by=ring,criticality&kev_onl
```

Target benchmarks: KEV critical + auto-deploy → <2h; KEV + approval → <24h; Non-KEV high → <72h; Standard → <14d

Urgency Score Formula

```
base = (cvss / 10) * 40  
kev_mult = 2.0 if in_cisa_kev else 1.0  
epss_pts = epss_score * 20  
overdue_pts = min(15, days_overdue // 7 * 3)  
criticality_m = {critical: 1.5, high: 1.25, standard: 1.0, low: 0.75}[device.crit  
exposure_m = max(EXPOSURE_MULTIPLIERS.get(t, 1.0) for t in device.tags) if device  
urgency_score = min(100, int((base * kev_mult + epss_pts + overdue_pts) * critica
```

AI Layer Rules

```
CRITICAL_EVENTS = {  
    "policy.evaluated", "deployment.dispatched", "cert.revoked", "cert.enrolled",  
    "ai.external_call", "ai.blast_radius_override", "exposure.accepted_risk",  
    "user.role_changed", "org.settings_changed",  
}  
  
# These BLOCK on audit INSERT failure. Action must not proceed.  
  
# AI is disabled by default. Explicit opt-in required per org.  
# Before every Claude API call:  
# 1. Check ai_policy.external_ai_enabled  
# 2. Write audit_log entry with payload_hash (this write must succeed first)  
# 3. Only then make the API call  
# Blast radius: nl_blast_radius_limit default 200 devices  
# Claude model: claude-sonnet-4-20250514
```

GDPR + NIS2 Compliance Requirements

GDPR:

- EU data residency: Frankfurt only
- DPA template required before customer onboarding
- Right to erasure: all org data purged within 30 days via `deletion_requests` workflow
- Data portability: export all org data as JSON/CSV
- Sub-processors: Supabase, Fly.io, Upstash, Anthropic (AI only, opt-in only)
- Breach notification: 72h to supervisory authority (Article 33)
- Anthropic AI = cross-border transfer → requires explicit org consent
(`external_ai_enabled: true`) + audit trail

NIS2:

- Customers in essential sectors (water, energy, healthcare) must comply with Article 21
- PatchPilot’s MTTRem report + audit log = NIS2 evidence
- `nis2_incidents` table with auto-computed deadline columns:
 - `early_warning_due` = `detected_at` + 24h
 - `notification_due` = `detected_at` + 72h
 - `final_report_due` = `detected_at` + 3 months
- `compliance_evidence` DB view for auditor export

Dashboard Pages

Route	Purpose
<code>/dashboard</code>	Fleet overview: KEV KPIs, MTTRem p50, top 20 urgent, ring status, feed health
<code>/dashboard/exposures/kev</code>	Active KEV CVEs, affected devices, deploy actions
<code>/dashboard/exposures/zero-day</code>	Open UnpatchedExposures, mitigations, recheck countdown
<code>/dashboard/deployments</code>	Ring rollout progress, job states, WebSocket live feed, anomaly halt banner

/dashboard/fleet	Device inventory, search/filter, per-device detail
/dashboard/intel	Feed health, staleness alerts, parse error indicators, blob provenance
/dashboard/analytics	MTTRem trend charts, KEV vs overall, ring/criticality breakdowns
/dashboard/compliance	NIS2 incident management, GDPR deletion requests, evidence export
/dashboard/settings	Patch policy, notifications, AI policy, enrollment tokens, audit log

Project File Structure

```

patchpilot/
├── backend/
│   ├── app/
│   │   ├── main.py           # FastAPI app + HardHeaderStrip ASGI wrapper
│   │   ├── config.py        # pydantic-settings
│   │   ├── database.py      # Async SQLAlchemy, pool_size=20
│   │   ├── models/         # ORM models (all inherit TenantMixin)
│   │   ├── schemas/        # Pydantic schemas
│   │   ├── routers/        # FastAPI routers (all use get_org_scope())
│   │   ├── dependencies/    # get_current_user(), get_org_scope(), get_mtls_
│   │   ├── middleware/     # MTLShHeaderGuard, HardHeaderStrip
│   │   ├── services/       # pki, audit, normalization, ai, notifications,
│   │   └── workers/        # intel_fetcher, vuln_matching, job_runner, zero
│   ├── migrations/         # Alembic
│   ├── tests/
│   │   └── intel/fixtures/  # Pinned sample blobs for parser contract tests
│   ├── pyproject.toml
│   ├── Dockerfile
│   ├── fly.api.toml
│   ├── fly.worker.toml
│   └── fly.beat.toml
├── agent/
│   ├── windows/
│   │   ├── main.py
│   │   ├── inventory.py     # OS identity, registry apps
│   │   ├── kb_collector.py  # Dual-method + cache
│   │   └── telemetry.py     # Per-job snapshot: CPU, crash events, reboots,

```

```

|   |   |─ patch_executor.py    # Winget + fallback chain
|   |   |─ checkin.py          # mTLS loop, BLPOP long-poll, delta hashing
|   |   |─ tests/
|   |   |─ pyproject.toml
|─ frontend/                    # Next.js (Vercel)
|   |─ app/
|   |─ components/
|   |─ package.json
|─ supabase/
|   |─ config.toml              # Local dev config
|─ .github/
|   |─ workflows/
|       |─ test.yml
|       |─ deploy.yml
|       |─ release.yml          # Agent build + sign + upload
|─ vercel.json
|─ .env.example                 # Secret names only, no values
|─ Makefile
|─ CLAUDE.md                    # This file

```

Build Phase Progress

Phase 0 — SaaS Skeleton (Session 0-new)

- Fly.io app configs (api, worker, beat) for Frankfurt
- GitHub Actions: test → build → deploy pipeline
- Vercel config with downloads CDN rewrite
- Supabase local dev config
- `.env.example` with all secret names
- `TenantMixin` base model
- `CLAUDE.md` updated with all invariants

Phase 1 — Secure Foundation (Sessions 1A–1D) · Weeks 1–4

Deliverable: Agents checking in via mTLS

- 1A: DB schema + core auth (Supabase JWT validation, compliance tables)
- 1B: PKI + mTLS + enrollment (envelope-encrypted CA keys, two-part tokens, `HardHeaderStrip`)

- 1C: Windows agent (inventory delta hashing, dual KB collection, telemetry snapshot)
- 1D: Deployment packs (versioned download URLs at downloads.patchpilot.com)

Phase 1 Gate: Agent enrolls → cert revocation tested → Redis down = DB fallback works → audit_log UPDATE raises error → backend not reachable directly

Phase 2 — Intel Pipeline (Sessions 2A–2D) · Weeks 5–8

Deliverable: Live KEV/MSRC feeds, urgency scores, zero-day workflow

- 2A: v2 schema (signal_ingested_at, mttrem_hours generated column, unpatched_exposures, nis2_incidents)
- 2B: Feed workers (KEV/MSRC/EPSS/NVD/GHSA, MSRC resilience, parser contract tests, Supabase Storage for blobs)
- 2C: Normalization v2 + vuln matching (OS path + third-party path separate)
- 2D: Zero-day response workflow (UnpatchedExposure entity, recheck loop, “response” not “detection”)

Phase 2 Gate: KEV entry → fleet exposure check → unpatched KEV → UnpatchedExposure created → recheck → transitions to patched when KB appears

Phase 3 — Smart Deployment (Sessions 3A–3C) · Weeks 9–13

Deliverable: Ring rollout with anomaly halt + dashboard

- 3A: Job state machine + ring rollout + anomaly halt (telemetry data now available from 1C)
- 3B: BLPOP command delivery + WebSocket live feed + fast cadence
- 3C: Dashboard API + NIS2/GDPR compliance endpoints + MTTRem metrics

Phase 3 Gate: Canary >20% failure → halt before pilot → fast cadence reaches agent <35s → MTTRem computes correctly → WebSocket live in <3s

Phase 4 — AI Intelligence Layer (Sessions 4A–4B) · Weeks 14–18

Ship after first paying customer. AI is the upsell, not the wedge.

- 4A: Governed AI layer (GDPR consent gate, redaction, blast-radius guard, audit-before-call)
- 4B: MTTRem executive report + AI narration + SSO stub + load prep

Phase 4 Gate: AI disabled by default → redaction removes hostnames → audit log written before API call → blast radius blocks non-admin at 201 devices

Current State

```
Session 0-new:  [ ] Not started
Session 1A:     [ ] Not started
Session 1B:     [ ] Not started
Session 1C:     [ ] Not started
Session 1D:     [ ] Not started
Session 2A:     [ ] Not started
Session 2B:     [ ] Not started
Session 2C:     [ ] Not started
Session 2D:     [ ] Not started
Session 3A:     [ ] Not started
Session 3B:     [ ] Not started
Session 3C:     [ ] Not started
Session 4A:     [ ] Not started
Session 4B:     [ ] Not started
```

What’s built: Nothing yet. Starting from scaffold.

Common Traps — Read Before Each Session

Trap	Prevention
CA keys in plaintext	<code>ca_key_encrypted</code> BYTEA only. <code>ca_key_pem</code> column does not exist.
NVD cadence changed to hourly	NVD = 6h by design. See decisions log. Test verifies Celery Beat schedule.
Agent JWT added	Agents use mTLS only. No JWT. Never add agent JWT.
Detection language	“Zero-day response” everywhere. Never “zero-day detection.”
Mutable audit_log	INSERT only at DB level. Never add cleanup job.
2s polling loop for commands	BLPOP only.

Full inventory every check-in	Hash delta. Full send daily at 03:00 only.
AI without consent gate	Check <code>external_ai_enabled</code> , write audit log, then call API. In that order.
Cross-org query	Every query: <code>WHERE org_id = scope.org_id</code> . Always.
Backend public port	Never. Fly.io handles proxy.

Recovery Prompts

If something is broken:

Read CLAUDE.md. Read the failing test in full. Do NOT propose a fix yet. Tell me: (1) which invariant was violated, (2) where in the codebase, (3) the minimal fix. I will confirm before you change anything.

If a session produced bad code:

List every file added or modified in the last session. List every test added. Run all tests. Show me pass/fail. Do not add new code until I confirm.

Moat verification (before any demo):

Verify all five moat primitives: (1) Add a KEV CVE → fleet check fires. (2) Deploy → audit log contains policy snapshot before dispatch. (3) `SELECT mttrem_hours FROM device_vulnerabilities WHERE patched_at IS NOT NULL` → non-null. (4) `UPDATE audit_log SET result='tampered'` → `PermissionError`. (5) KEV CVE with no remediation → `UnpatchedExposure` row, not `NULL` `device_vulnerability`. Report pass/fail for each.

Update "Current State" after every session. Keep this file under 400 lines — summarise older sections if it grows.